heatedPool — internally heated liquid pool with a free surface (OpenFOAM v2606)

Case generator for natural circulation in a small closed container of liquid with uniform volumetric heat generation, heat loss through the side wall and a flat free surface on top. Single phase, Boussinesq, laminar (`buoyantBoussinesqSimpleFoam` / `buoyantBoussinesqPimpleFoam`).

Geometry	Domain	Mesh (blockMesh)
A	annulus, D_i 5 mm, D_o 8.6 mm, H 75 mm	4 × 90° blocks, radial × azimuthal × axial
B	cylinder, D 7 mm, H 35 mm	curved O-grid (square core + 4 outer blocks)

File	Purpose
`flowAnalysis.py`	Natural-circulation plots, contours and summary for a finished case
`meshStudy.py`	Mesh-sensitivity study on wedges with Grid Convergence Index
`setupCase.py`	**Run this.** Asks the questions (Enter accepts the suggestion), writes one case into `cases/`
`caseDefaults.py`	Suggested answers + settings that are not asked (salt and cladding properties, calibration targets, probe layout, solver controls)

```
python3 setupCase.py                      # questionnaire
python3 setupCase.py --defaults           # straight from caseDefaults.py
python3 setupCase.py --from cases/<case>/caseSettings.py   # replay / tweak an earlier case
cd cases/<case> && ./Allrun
```

Check before running

Setting	Default	Status
Salt heat generation	20 W/g (`SALT_POWER`) × salt mass → 87.3 W/cm^3 at T_{ref}, Q_{gen} = 252 W (A), 118 W (B)	set
Salt ρ, c_p, k, β	user data at T_{ref} = 946 K: 4365 kg/m^3, 924 J/kg·K, 1.16 W/m·K, 1.83×10^{-4} 1/K	set
Salt viscosity	μ = 0.307 mPa·s · exp(21138/(RT)), temperature dependent (4.51 mPa·s at 946 K)	set
Cladding	Hastelloy N (k(T) table, ε 0.25, ρ 8860 kg/m^3), 17 mm OD for A and B, 5 mm bottom plate, A inner ring 2.5 → 5 mm; **2 W/g heat generation** (17.7 W/cm^3)	not modelled yet: the cladding above the liquid level
Wall BCs	`cladding` with `outside = "measured"` (default): outer-wall and bottom-plate conduction with generation to the measured outer-surface temperatures (A: T-4…T-12, B: T-18…T-23, θ-averaged spline); A inner wall `ring`; top adiabatic. Alternatives: `split`, `cladding` with gas. Boundary-layer mesh on	which plateau matches 20 W/g (`TC_PLATEAU`); T-10 excluded (`TC_EXCLUDE`)
Radiation in the liquid	off. Options P1 / fvDOM with a grey absorption coefficient	a: **placeholder**

- At 100 W/cm^3 the estimated thermal boundary layer is ~0.8 mm (A) and ~0.3 mm (B) with the salt properties. Use the boundary-layer mesh, especially for B: without it, B (medium) has a single cell across the layer.

Physics and boundary conditions

Patch	U	T	p_rgh
`outerWall`	noSlip	split (default: shaped flux) / cladding (coded BC, measured outer T or gas) / coefficient / balance / flux	fixedFluxPressure
`innerWall` (A)	noSlip	ring (default `impose_tc = True`: fixed at the axis thermocouples; or the ring's own heat as a flux) / adiabatic / coefficient / flux	fixedFluxPressure
`bottom`	noSlip	relative (default: mean flux 0.34 × the outer wall's) / split / cladding (5 mm plate) / adiabatic / flux	fixedFluxPressure
`freeSurface`	**slip** (flat, shear-free)	relative (default: mean flux 0.17 × the outer wall's = 0.5 × bottom) / split / adiabatic / coefficient / flux	fixedFluxPressure

With the thin-shell wall model (`WALL_MODEL = "shell"`) the three cladding walls are merged into one patch, `cladding`, and kept as faceZones `innerWall`, `outerWall` and `bottom`. The `cladding` and `ring` modes need `WALL_MODEL = "1D"` (default).

- **Free surface:** a flat, impermeable, shear-free lid replaces the liquid–gas interface, so no second phase is needed. This is valid while the surface stays flat (it does at these velocities). Not included: Marangoni stress (dσ/dT). At mm scale it can compete with buoyancy, so keep it in mind.
- **Salt power per mass:** `SALT_POWER` = 20 W/g, so $q''' = P_m \cdot \rho(T)$. With `POWER_FOLLOWS_DENSITY` the Boussinesq T source is $P_m/c_p \cdot (1 - \beta(T - T_{ref}))$: explicit + implicit part of `scalarSemiImplicitSource` (specific). Hotter salt generates slightly less ($\beta \cdot \Delta T \approx 2$ % over 100 K); `heatBalance` integrates the actual generation.
- **Heat source:** `scalarSemiImplicitSource` on T, `volumeMode absolute`. The Boussinesq T equation is kinematic, so the script writes the total $q''' \cdot V/(\rho c_p)$ in K·m^3/s.
- **Viscosity μ(T):** `ArrheniusNewtonian` ($\nu = \nu_{ref} \cdot \exp(-\alpha(T - T_{ref}))$, with $\alpha = E_a/(R \cdot T_{ref}^2) = 2.84 \times 10^{-3}$ 1/K). It is the tangent of $\mu_0 \cdot \exp(E_a/RT)$ at T_{ref}, within 1 % over ±60 K. The solver would also scale the thermal diffusivity (ν/Pr) with ν, so a coded fvOption (`constantConductivity`) adds $\nabla \cdot ((\alpha_0 - \nu(T)/Pr)\nabla T)$ and k stays 1.16 W/m·K.
- **Heat loss:** `externalWallHeatFluxTemperature` with `kappaMethod function` (constant k), because the Boussinesq solvers have no thermo object. The library is loaded with `libs (thermoTools)`. The flux and balance modes are written as `mode power` (a total in W, spread uniformly). For flux, the input is `q_loss` (positive = heat out); the script writes $Q = -q_{loss} \cdot A$.
- **Closed domain:** the pressure level is fixed with `pRefCell 0; pRefValue 0`.

Temperatures where measured, small fluxes where not (default)

Only temperatures are measured, so they are imposed where they exist and fluxes only where they do not:

Wall	Condition
A inner wall	fixed at the axis spline (T-3/T-2/T-1) − 0.49 K ring offset (`ring`, `impose_tc = True`)

Wall	Condition
outer wall (A, B)	conduction with generation through the cladding to the outer-surface spline (`cladding`, `measured`)
bottom	mean flux 0.34 × the outer wall's mean flux (`relative`; reference model, per area)
free surface	mean flux 0.17 × the outer wall's (= 0.5 × bottom, user data)

Per-area ratios already scale with the exchange surface. A's annulus bottom (38.4 mm^2) and B's disc (38.5 mm^2) happen to be the same size, and both give ~1.7 W through the bottom. The ring heat is then an output: physically it must be +16.6 kW/m^2 (19.6 W) into the salt; `postProcessing/innerWallT` writes T and grad(T) at the thermocouple heights (heat into the salt there = $-k_{salt}\,\partial T/\partial x$), and `heatBalance` the total. Where it differs, physics is missing.

Result (A, medium mesh with boundary layer, plateau 1, converged). With the axis profile imposed, the salt takes only **1.6 W** from the ring (heat balance) instead of the 19.6 W the ring generates. Along the height (31 samples at the inner wall): the ring would have to take heat *out* of the salt over the lower 30 mm (down to −39 kW/m^2) and give it *more* than its own heat in the upper half (up to +24 kW/m^2 against +16.6). The measured axis temperature rises much more gradually from the bottom (612 → 680 °C over 30 mm) than the salt next to it wants. A Hastelloy ring cannot move that much heat along itself (~9 W would need ~27 K/mm against 2.4 K/mm measured), so the mismatch points at the heat *source* or the readings: an axial power profile (lower generation towards the bottom), thermocouples reading below the ring (sheath conduction, gap), or a stronger bottom heat loss. B (converged in 1692 iterations): bottom 1.9 W, top 1.0 W; axis 676 / 726 / 784 °C (no internal readings).

Thermocouples and measured outer walls

`TC_READINGS` holds the steady readings [°C] of three plateaus (all thermocouples step up together; `TC_PLATEAU` selects one). `TC_LAYOUT` maps them: A axis T-3/T-2/T-1 (bottom/middle/top, inside the inner ring), A outer surface T-4…6 / T-7…9 / T-10…12 (bottom/middle/top, different θ), B outer surface T-18 / T-19, T-20 / T-23. Readings at one height are averaged over θ; T-10 reads 70–100 K below T-11/T-12 and is excluded (`TC_EXCLUDE`). Plateau 1: A axis 612 / 690 / 711 °C, A outer 554 / 565 / 555 °C, B outer 632 / 618 / 603 °C.

The outer wall and bottom plate use mode `cladding` with `outside = "measured"`: conduction with generation through the Hastelloy to the outer-surface spline (the plate's far side at the bottom reading). No gas parameter and no level calibration: the energy balance closes by itself, and for A the axis thermocouples are a genuine check. The free surface loses heat with mode `relative`: its mean heat flux is `ratio` (0.5, user data) × the bottom's mean heat flux, read from the solution every iteration by a small coded BC. The inner wall of A stays `ring`: no-slip, with the ring's own heat into the salt. A symmetry condition would make the salt slip along the ring and drop that heat; with radiation off the ring's view of the salt plays no role.

Result (coarse mesh with boundary layer, plateau 1, 1000 SIMPLE iterations, probes converged). Exact balances (A: 252.3 W salt + 19.6 W ring = 270.3 W side + 1.7 W bottom; B: 116.5 W = 114.9 W side + 1.7 W bottom). A axis, model against measured: 586 / 707 / 748 °C against 612 / 690 / 711 °C (−26, +16, +37 K). B (no internal readings): axis 676 °C at the bottom, 727 °C mid-height, 796 °C at the top. Sensitivity: with T-10 included (A outer top 527 instead of 555 °C) the A axis top drops to 729 °C (+18 K); bottom and middle are unchanged (−26, +18 K). The remaining error is an over-stratified salt: too cold at the bottom, too hot at the top. Candidates: mesh (coarse), radiation inside the salt, heat loss through the free surface, 3D/azimuthal effects.

Medium mesh with boundary layer + top heat loss (`relative`, 0.5 × bottom flux), plateau 1, T-10 excluded. A (407k cells): balance exact; bottom 1.69 W, top 0.85 W. Axis thermocouples, model against measured: 586 / 706 / 739 °C against 612 / 690 / 711 °C (−26, +16, +28 K). Bottom and middle are within 0.1 K of the coarse mesh (mesh-independent); the top dropped by ~10 K (top loss). B (185k cells, converged in 1681 iterations): axis 676 °C near the bottom, 726 °C at mid-height, 784 °C near the top (−11 K from the top loss). The remaining A error is not a mesh effect. It sits at the two ends, where the model idealises the ring: at z = 0 the real ring stands on the 5 mm bottom plate (conduction into the cooler plate lowers the bottom thermocouple), and at the top the ring continues above the liquid into the plenum. Both lower the measured end temperatures.

Heat split from a reference model (mode `split`)

`referenceProfiles.py` reads a reference model of a similar capsule (SALIENT-03, `Salt surface temp and flux.xlsx`: R 3.8 mm, H 36 mm) and writes `analysis/reference_profiles.png` and `analysis/reference_shapes.csv`. Its findings:

- the bottom releases only 1.8 % of the heat, at a mean flux 0.34 × the side wall's;
- the side flux is 0.7–0.9 × its mean over most of the height, 1.5 × at 0.9 H and 3.6 × at the top edge: the top 10 % of the height releases 21 % of the side heat (the side wall near the top is strongly cooled);
- its normalised wall-temperature shape matches the A axis thermocouples (θ = 0, 0.77, 1 against 0, 0.78, 1).

`split` takes only these shapes and ratios. The heat leaving the salt (salt generation + ring heat) is shared by mean-flux ratios that transfer between geometries: bottom = `bottom_over_side` × side (0.34), top = `top_over_bottom` × bottom (0.5, user data). The side wall gets the reference shape $q/q_{mean}(z/H)$ (coded fixed-gradient BC); bottom and top are uniform (power mode). Every total is normalised on the mesh, so the energy balance is exact. With flux-only boundaries the level is free: `LEVEL_PIN` shifts T after every T solve so that the A inner-wall mean equals the axis thermocouple spline mean minus the ring offset (`innerWallTC`), or the volume mean equals T_{init} (`volume`, B until there are readings). The thermocouple *shape* is then a genuine prediction.

The reference top zone (its top 10 %, 3.6 mm: cooling by the cladding above the liquid) is kept at its absolute thickness (`HEAT_SPLIT top_zone`); the rest of the shape is stretched over the height. For B this is the reference as is; for A the top 3.6 mm then carry 10.7 % of the side heat instead of ~21 %.

Result for A (coarse mesh with boundary layer, 700 SIMPLE iterations, residuals ~1e-4, probes stable). Exact balance: 251.8 W salt + 19.6 W ring = 268.7 W side + 1.7 W bottom + 0.9 W top. Inner wall at the thermocouple heights: 962 / 969 / 888 K against 893 / 963 / 983 K (minus the 0.5 K ring offset): hot bottom, cold top, the reverse of the measurement. The reason is the annulus itself: the inner wall averages 953.5 K and the outer wall 812.7 K, 141 K across a 1.8 mm gap (pure conduction would give ~206 K). Heat crosses A's gap mostly by conduction, so each height loses its own heat locally and the axial profile is set by the cooling *distribution* on the outer wall, not by convection in the salt. The reference flux shape (low at the bottom, high at the top) belongs to a convecting Ø7.6 mm cylinder; on A it inverts the profile. The measured A profile instead says the outer wall is *colder at the bottom*. For B (a cylinder like the reference) the split stays the natural choice. For A, see the cladding mode with an assumed outer profile.

Cladding with heat generation (modes `cladding` and `ring`)

The Hastelloy cladding generates heat: q_c = 2 W/g × 8.86 g/cm^3 = 17.7 W/cm^3. That is 224 W in A's outer cladding (117 W in B), 20 W in the inner ring and 20 W in each bottom plate, of the same order as the salt (288 W in A, 135 W in B). `claddingAnalysis.py` prints these numbers and the axial-conduction check.

`cladding` **(outer wall, bottom): a coded Robin condition.** Steady radial conduction with generation, (1/r) d/dr(k r dT/dr) + q_c = 0, through the tube $r_i \to r_o$ (or a plate of thickness t) gives

```
q_liquid = H (T_w - T_ref)      [W/m2 of liquid-side wall]

h_c    = k / (r_i ln(r_o/r_i))                      plate: k/t
dT_gen = q_c/(4k) [r_o^2 - r_i^2 - 2 r_i^2 ln(r_o/r_i)]
                                                    plate: q_c t^2/(2k)
outside = "measured" (thermocouples on the outer surface):
    H = h_c,  T_ref = T_o(z) + dT_gen
outside = "gas":
    q_o r_o = q_i r_i + q_c (r_o^2 - r_i^2)/2
    T_o = T_a + q_o/h_g,  h_g = h + h_rad(T_o)
    H = 1/(1/h_c + r_i/(r_o h_g))
    T_ref = T_a + dT_gen + q_c (r_o^2 - r_i^2)/(2 r_o h_g)
```

The `codedMixed` BC evaluates this per face, every iteration: `refValue` = T_{ref}, `valueFraction` = H/(H + k_{salt}·δ). It uses k(T) of Hastelloy N at the mean cladding temperature, and the radiation at the current outer-surface T. For A's outer wall, dT_{gen} = 5.7 K, for B 7.7 K, for the plate 9.9 K.

`ring` **(A inner wall).** The ring is closed inside, so at steady state its own 19.6 W can only go into the salt: a fixed flux of $q_c(r_b^2 - r_a^2)/(2r_b)$ = 16.6 kW/m^2. Conduction across the ring puts the axis thermocouple 0.49 K above the inner wall. Both the flux and the temperature are fixed by the ring, so only one can be imposed: `impose_tc = False` (default) imposes the flux and leaves the thermocouples for validation; `impose_tc = True` imposes the thermocouple profile (spline − 0.49 K), and the flux into the salt then comes out of the run (check it against 16.6 kW/m^2).

Thermocouple profile. `THERMOCOUPLES` (T in K at z in m) and `TC_PROFILE`, via `tprofile.py`: `clamped_top` (cubic spline, dT/dz = 0 at the free surface, default), `natural` or `pchip` (monotone). `analysis/A_profile.png` shows the three for A; they agree within ~3 K.

First result (A, coarse mesh with boundary layer, 677 iterations to convergence, T_a = 300 °C, dT_internal = 0). Heat balance exact (Q_{gen} = Q_{out} to 0.002 W; the ring gives 19.6 W to the salt, the outer wall removes 307 W). Axis thermocouples (inner wall + 0.5 K): 914 / 1059 / 1180 K against 893 / 963 / 983 K. The spread is 267 K against 90 K: the ring heat adds to the hot layer under the adiabatic free surface. The second pass (dT_internal = 109 K) shifts the level, not the shape. Candidates for the missing mixing / heat path: the cladding and gas plenum above the liquid (top not adiabatic), radiation inside the salt, and unsteady mixing (check with the transient stage).

Heat removal through the cladding (mode `coefficient`, no generation)

The `coefficient` mode (no cladding generation) models the path: liquid → conduction through the cladding (`layers`: thickness, k) → outer cladding surface → gas convection h and radiation εσ(T_s^4 − T_a^4). OpenFOAM solves for the outer-surface temperature T_s face by face, so a hotter part of the wall loses more heat. This is the feedback that limits the vertical stratification. Note that T_a is used both for the gas and for the radiating surroundings.

Calibrating the unknown h (`h = "auto"`). At steady state the losing walls (outer wall and bottom plate, which share the gas h and T_a through `h = "outerWall"`) must carry all of Q_{gen}. Given the cladding, ε and T_a, the script finds by bisection the h that removes Q_{gen} with the walls at the

temperature implied by `T_target`:

`target`	**the measured temperature is…**
`liquid`	the mean liquid temperature
`innerWall`	A: the inner wall. The thermocouples on the axis inside the adiabatic inner cladding ring read this temperature
`wall`	the cladding, liquid side
`surface`	the cladding outer surface

For `liquid` and `innerWall`, the measured point is hotter than the losing walls by an amount only the CFD knows (`dT_internal`):

- 1. run with `dT_internal = 0`. The losing walls then sit at T_{target};
- 2. read T_{avg} − Tw_loss (`liquid`) or $T_{innerWall}$ − Tw_loss (`innerWall`) from `postProcessing/heatBalance_*.dat` (Tw_loss = area-mean T of the losing walls), set it as `dT_internal`, regenerate and rerun. It hardly depends on h, so one correction is usually enough. For A it is 87 K (thin shell, coarse mesh).

The fitted level (mean T) is then imposed. The test of the model is whether the axial profile comes out right. For A, `postProcessing/innerWallT` samples the inner-wall temperature at the thermocouple heights (z = 0, 37.5, 75 mm; measured 620, 690, 710 °C).

Curved cladding. Both wall models treat the cladding as flat, with h and the radiation on the liquid-side area. The script writes the exact flat equivalent of a cylindrical cladding: each thickness becomes $r_w \cdot \ln(r_{out}/r_{in})$, and h and ε are multiplied by $r_{surface}/r_w$. For B (3.5 → 8.5 mm) that is a 3.1 mm equivalent thickness and a factor 2.43.

Thin-shell wall model (`WALL_MODEL = "shell"`)

The one-dimensional wall condition (`WALL_MODEL = "1D"`) conducts only through the wall. The Hastelloy cladding also conducts **along** the wall: the A inner ring alone carries heat axially about 7× better than the salt annulus. The shell model (`regionFaModels thermalShell`) solves a 2D conduction equation on an area mesh of the cladding walls:

- `blockMesh` → `topoSet` (cladding walls as faceZones) → `createPatch` (merged into `cladding`) → `makeFaMesh` → `./setShellThickness`, which sets the per-wall shell thickness from `faceLabels` and the faceZones.
- Shell T = liquid-side wall T (`compressible::thermalShell` BC, fixed value on the liquid side). `contactHeatFluxSource` feeds in the liquid's wall flux.
- Shell thickness = cladding thickness × r_{mean}/r_w, so the axial conductance and heat capacity equal those of the thick tube: inner ring 0.94 mm, A outer 6.25 mm, B outer 8.57 mm, bottom plate 5 mm. The bottom plate is joined to the walls at the corners.
- `externalHeatFluxSource` (per faceZone) removes heat with one coefficient per wall area: the through-wall resistance in series with gas convection and radiation (linearised at the outer-surface T).

Pitfalls in v2606 that the script avoids:

Item	**Problem**	**Done instead**
one shell region on several wall patches	each patch BC creates its own shell; the contact source only feeds the first one	walls merged into one patch, faceZones for per-wall data

Item	Problem	Done instead
`faSolution nNonOrthCorr` > 0	the contact source adds its term only to the first shell solve of a time step, so the final solve runs without the liquid's heat (walls end up at T_a)	`nNonOrthCorr 0`
`contactHeatFluxSource thicknessLayers`	the layer conductance is added in parallel to the wall conductance (`htc + 1/R`), not in series	not used; resistance on the external side
`externalHeatFluxSource emissivity`	the radiation term adds $+\varepsilon\sigma T^4$ to the shell (a heat gain)	not used; radiation linearised into h

What the unknown gas temperature does (A, coarse mesh, 1000 iterations, dT_internal = 87 K)

With h re-calibrated for each T_a, T_a sets how strongly the loss reacts to the local wall temperature: a hot gas needs a large h, which gives strong feedback and a flatter profile. Inner-wall T at the thermocouples:

T_a	h (calibrated)	z = 0	37.5 mm	75 mm	spread
573 K (300 °C)	175 W/m^{2}K	832 K	959 K	1014 K	182 K
773 K (500 °C)	1020 W/m^{2}K	851 K	963 K	995 K	143 K
823 K (550 °C)	~4000 W/m^{2}K	859 K	966 K	989 K	130 K
823 K, adiabatic bottom	4000 W/m^{2}K	863 K	966 K	986 K	123 K
measured		**893 K**	**963 K**	**983 K**	**90 K**

- Above ~830 K no h works: the 4.2 mm cladding alone cannot pass 288 W with the walls at ~860 K.
- The top half matches at high T_a (+23 K vs +20 K measured), but the bottom stays 30–60 K too cold. The bottom plate is not the cause: making it adiabatic warms the z = 0 thermocouple by only 4 K.
- The h values at high T_a (1000–4000 W/m^{2}K) are far above gas convection. So either the real gas is cool and the liquid mixes more than the model predicts, or there is another loss path.
- Still to test: a finer mesh (the coarse mesh has only 2 cells across the thermal boundary layer, which tends to overpredict stratification) and radiation inside the salt (an extra effective conductivity of order k).

For comparison, the first 1D-wall run (1 mm placeholder cladding, probes at 1.5 and 73.5 mm) spread over ~234 K. The first shell run at T_a = 573 K spread over ~197 K: axial conduction in the cladding helps, but it is not enough on its own.

`balance` (a constant flux removing exactly Q_{gen}) is still available, but it has **no feedback**: a hotter region loses no more heat than a colder one. Stratification then grows unchecked, and with flux-only boundaries the mean T is pinned to T_{init} by a coded fvOption.

Radiation inside the liquid (`RADIATION`)

The liquid is treated as a grey, absorbing–emitting, non-scattering medium. The walls are opaque, grey and diffuse. The Boussinesq solvers already contain the radiation source term (`radiation->ST`). The script adds:

- `constant/radiationProperties`: model, `solverFreq`, `constantAbsorptionEmission` with the **absorption coefficient a** (the key input, as a grey mean over the spectrum where the liquid emits);
- `constant/boundaryRadiationProperties`: emissivity of each surface as seen from the liquid;
- `0/G` (P1, `MarshakRadiation`) or `0/IDefault` (fvDOM, `greyDiffusiveRadiation`), plus solver and scheme entries;
- `rhoRef`, `CpRef` in `transportProperties`, used to turn the radiative source into K/s;
- `qr qr;` in every wall T condition. The radiation absorbed by a wall enters its heat balance, and an adiabatic wall becomes re-radiating (net flow zero).

Model	Cost	Valid for
P1	one diffusion equation	optically thick, a·L ≫ 1
fvDOM	one transport equation per ray (4·nPhi·nTheta, 16 by default), solved every `solverFreq` iterations	any optical thickness

`caseSummary.txt` prints the optical thickness a·L and the optically thick (Rosseland) radiative conductivity $16\sigma T^3/(3a)$, to compare with k. At 670 °C, a = 100 m^{-1} gives about 2.5 W/m·K, twice the salt's k of 1.16. Two limitations of the radiation model: emission is σT^4, i.e. refractive index 1 (a liquid with index n emits n^2 times more), and the medium is grey.

Run modes

RUN_MODE	What Allrun does
`steadyThenTransient` (default)	SIMPLE to a steady state → copy it to `0/` (kept in `steadyResult/`) → PIMPLE for `TRANSIENT_END_TIME` to confirm the flow really is steady
`steady`	SIMPLE only
`transient`	PIMPLE from rest, e.g. to reproduce a heat-up. `caseSummary.txt` gives the thermal time constant: run ~3–5 of them

SLURM cluster (`CLUSTER`, questionnaire section 8). With `enabled=True` the case gets `Allrun.slurm` instead of `Allrun`; submit it from the case directory with `sbatch Allrun.slurm`. The script:

- has a header of `#SBATCH -n <ntasks>`, `#SBATCH -A <account>` (left out when empty) and any `extra` options (e.g. `["-t 48:00:00", "-p cpu"]`); - runs `cd "$SLURM_SUBMIT_DIR"` and adds `$FOAM_USER_APPBIN` to `PATH`; - sets `numberOfSubdomains` from `$SLURM_NTASKS`; - runs the solvers (and `makeFaMesh` for the thin shell) with `srun <app> -parallel > log.<app>.<stage>`, and the serial utilities with `runApplication`; - keeps the same stage logic as Allrun (steady → move to 0 → transient, particle fvOptions swap); - ends with `reconstructPar -newTimes` and `touch foam.foam`.

`ntasks` replaces `N_PROCS`; wedges take whatever is set (the questionnaire suggests 1, or 4 with particles; with 1 there is no decomposition). The OpenFOAM environment comes from the submitting shell (sbatch exports it by default), as in the HXThermalInlet scripts.

Starting from a finished case (`INITIAL_FIELDS`). Set `INITIAL_FIELDS = "cases/<finished case>"` (or answer the questionnaire question in section 6) to start, e.g., a `transient` particle run from a converged steady case without repeating SIMPLE. The generator then:

- checks the mesh is identical (same generated `blockMeshDict`; otherwise use `mapFields`) and needs a reconstructed, ascii case; - writes the internal fields of its latest time into `0.orig`, so `Allrun/restore0Dir` keep the steady start; - keeps this case's boundary conditions but takes over their state (`value`, `refValue`, `refGradient`, `valueFraction`, and the `fixedFluxPressure` gradient) when the

patch type matches; - copies `phi` and `p` unchanged.

The patch state matters. Mixed BCs are evaluated from refValue/refGradient/valueFraction when read, and fixedFluxPressure needs its gradient for the hydrostatic balance. With placeholders instead, the first step starts from wrong wall values: the first Ux residual was 0.86 against 1.5e-5 with the full state. `caseSummary.txt` records the source and time. The transient wedge cases `*_TR_wedge` have been converted: their `0.orig` now holds the steady fields they started from.

Transient check (medium BL wedges, 2026-10-12). The converged SIMPLE fields of A and B were run on with buoyantBoussinesqPimpleFoam for 120 s (`cases/*_medium_BL_TR_wedge`; Co ≤ 0.8, Δt ≈ 0.02 s, about 5 A turnovers and 10 B turnovers). Over the whole run the probe temperatures moved by < 3 mK and the probe velocities by < 0.2 µm/s, and the heat balance stayed at Qgen − Qout ≈ −0.5 mW. The steady state holds. A wedge cannot develop 3D instabilities, so the definitive check is the 3D `steadyThenTransient` case (on the cluster).

Lagrangian fission-product particles (`PARTICLES`)

A one-way coupled nuclear particle cloud (`FPCloud`, NuclearCloud from `src/myIntermediate`) rides on the transient stage. It is attached through the fvOption `nuclearCloudSourceSingle` (`src/nuclearCloudFvOptions`, `libs (... nuclearCloudFvOptions)` in the transient controlDict). The cloud is evolved once per time step from `fvOptions.correct(U)`, so it needs `momentumPredictor yes` and `nOuterCorrectors 1` (both set). It never changes U or T. With `steadyThenTransient`, Allrun swaps `constant/fvOptions.{steady,transient}` so that the cloud exists only in the PIMPLE stage.

Setting	Value
injectors	one `manualInjection` per diameter (default 1e-9, 1e-8, 1e-7, 1e-6, 1e-5 m), `injectorID` 0, 1, ..., `n_per_injection` parcels each (default 10 000, nParticle 1), released one by one evenly over `release_duration` (default 2 s) from `SOI` (default 1 s into the transient stage); `release_duration = 0` releases them all at SOI. The cloud is only built at the 2nd time step, so SOI must be > 0
positions (`injection`)	`domain` (default): random (seeded), unique points uniform in the volume of the whole liquid, 0.25 mm off the walls (`constant/FPCloudPositions<i>`, manualInjection, exact number, gradual release). `central`: the same in the central region (`margin` = 0.25 of the gap/radius and of the height kept free next to every surface). `cellZone`: cellZoneInjection over the cellZone `liquid` (the whole mesh, made by `topoSet -dict system/topoSetDict.liquid` in Allrun): random points in every cell, number ≈ `n_per_injection` (rounded per cell), all released at SOI (no `release_duration`)
forces	sphereDrag, shear lift (`lift`: `McLaughlinMei` = Saffman × McLaughlin's J(ε)/J∞, Mei's fit, default; `SaffmanMei`; `none`), gravity (with buoyancy), BrownianMotion λ = 1e-12 (`brownian`, or BoundedBrownianMotion)
integration	U analytical (`u_integration`), T analytical; maxCo 0.5, logFrequency 1000
heat transfer	AhmedYovanovich (no T coupling back); no decay heat, dispersion, packing, damping or isotropy model
particle	ρ 10980, T0 883 K, cp 296, κ 115, µ0 = salt µ(T_ref)
carrier	ρ and ν constant at T_ref (the fvOption takes constants: the drag does not follow the Arrhenius µ(T), ±20 % over the salt)

Setting	Value
walls	multiInteraction: localInteraction (`stick` on every wall and on the free surface, `outputByInjectorId`, `writeToFile`) + stickReplaceInteraction: each stuck parcel is replaced by a new one (own ID, age 0) at a random point of the cellZone `liquid`, its hit (time, position, age, origProc, origId) is written to `postProcessing/lagrangian/FPCloud/patchInteractionModel/<startTime>/stickHits_<patch>_<typeId>.dat` at the write times, and the stuck parcel is deleted (`deleteStuck`, `writeHits`): the cloud stays at the injected size
cloud functions	`forceMagnitudes` (switch `force_magnitudes`; recomputes drag/gravity/Saffman-Mei lift with the constant carrier properties, not the forces applied), `patchInteractionFields` (cumulative deposition per face); `particlePostProcessing` (one record per hit) only when the walls are not `stick`

Wedges. The nuclear parcels keep a velocity component across the wedge, from the Brownian kicks and the lift (the 1 µm parcels of the smoke test reached U_y ≈ 4 mm/s and spread over the whole wedge angle), so they do reach the front and back faces. A wedge face that the patch interaction does not handle aborts the run ("Hitting a wedge patch should not be possible"). The faces therefore get an elastic `rebound` (e = 1, µ = 0). For an axisymmetric field this is exactly the 100 % recycling: leaving through one face and coming back through the other at the same (r, z) is the same as a mirror reflection. A random walk in a reflecting wedge also reaches the correct 3D equilibrium (density uniform in volume, ∝ r per unit radius). The parcels are injected within 90 % of the wedge angle. B's axis is a collapsed edge, so there the injection region also keeps 0.5 × margin × R off the axis. localInteraction counts only stick/escape, so rebounds on front/back do not appear in `localInteraction.dat`.

Smoke test (wedges, 1 µm + 10 µm, 1000 parcels each, 5 s). Both A and B run (15 s / 10 s CPU). Injection, stick and localInteraction/particlePostProcessing output work; one 1 µm parcel stuck on B's free surface.

Verification after the library fixes (wedges, all five sizes 1 nm–10 µm, 1000 parcels each, 5 s, stock BrownianMotion + analytical U, forceMagnitudes on). A runs in 97 s and B in 79 s of wall time (test cases since deleted). The cloud evolves once per time step, and FDrag/FGrav/FLift/FPg/FVm are written per parcel. With BoundedBrownianMotion + CrankNicolson, the same 1 nm/10 nm parcels did not get through the injection step in 30 min.

Fixed in the libraries (2026-10-12): `nuclearCloudSourceSingle` now evolves the cloud once per time step. buoyantBoussinesqPimpleFoam also calls `fvOptions.correct(U)` after every pressure corrector (`pEqn.H`), so before the fix the cloud advanced 1 + nCorrectors steps per time step. ForceMagnitudes is now registered for nuclear parcels.

Restarts (fixed 2026-10-13). `nuclearCloudSourceSingle` used to build the cloud at its first `correct()` call, when the time had already advanced past the restart time. The cloud then read nothing from the restart folder, started empty, and the injection models (no saved state) released every parcel again at once. The cluster run continued from 60 to 180 s showed this: new parcels with reused IDs, all with the same age, and the parcels stuck before the restart gone. The cloud is now built in the fvOption constructor, at the start time, so it reads the parcels (position, `active`, age, …) and `uniform/lagrangian/<cloud>` (the injection state).

Test (A wedge, 1 nm and 1 µm, 300 each; one run 0 → 6 s against 0 → 3 s plus a restart 3 → 6 s): at 6 s, the same 600 parcels with the same IDs, sizes and ages, the same stuck parcels (none moved), and no re-injection. Without Brownian motion the paths agree to 0.016 µm, the precision of the written files. With it, the random kicks after the restart differ (the random-number state is not saved), so the paths differ while the statistics are the same.

Rebuild `nuclearCloudFvOptions` (and `myIntermediate`) on the cluster before the next particle run. The post-processing notebook detects a cloud that was rebuilt at a restart (the same parcel ID with a different size) and then analyses only the time after it.

Brownian force and U integration for dt ≫ τp. In the salt, τp = ρp d² Cc/(18 μ) is 1.4e-13 s at 1 nm and 1.4e-5 s at 10 μm. Every parcel sub-step (~1e-3 s) therefore has dt/τp from ~10² up to ~10¹⁰. The parcel moves with the velocity it holds over the sub-step (`KinematicParcel::move`: `s = trackTime*U_`, then `calc`), so what sets the transport is the displacement over a step, not the velocity at its end:

- `BoundedBrownianMotion` makes the end-of-step velocity follow the exact Ornstein–Uhlenbeck law, so |U| ≈ v_th = √(kT/m), which is 48 m/s at 1 nm. Holding it over dt gives a diffusivity v_th²dt/2 = D·dt/(2τp) instead of D = kTτp/m. With the maxCo sub-stepping, dt shrinks to ~maxCo·Δx/v_th, and the diffusivity becomes ~0.14 Δx v_th: set by the mesh, ~10⁶ × D at 1 nm, and the run crawls. - The stock `BrownianMotion` (Li & Ahmadi), integrated analytically, gives U − Uc = √(2D/dt) ξ for dt ≫ τp. That is exactly the mean velocity of a diffusing particle over dt, giving a displacement √(2D dt): the Ermak–McCammon Brownian-dynamics step. Its dt^(-1/2) growth is the correct path scaling, not a divergence, and it reaches v_th only when dt → τp. With sub-stepping, dt_{k+1} = maxCo Δx/√(2D/dt_k) converges upwards (to (maxCo Δx)²/2D), so there is no runaway. - `CrankNicolson` for U has the amplification factor (1 − x/2)/(1 + x/2) → −1 for x = dt/τp ≫ 1. The slip is mirrored every step instead of damped, the Brownian kicks accumulate, and the velocity variance grows without bound. `analytical` (factor e^(−x)) is required in this regime.

A replay of the exact parcel update (`analysis/brownianCheck.py`, 4000 particles, 400 steps, still salt) gives the diffusivity ratio D_num/D:

d, dt	dt/τp	bounded + analytical	stock + analytical	bounded + CN	stock + CN
1 nm, 1 μs	7e6	3.7e6	**1.00**	1.8e6 (U std 923 m/s)	1.97
100 nm, 1 ms	7e5	3.7e5	**1.00**	1.8e5	1.96
10 μm, 1 ms	74	37	**1.00**	10	1.02
10 μm, 1 μs	0.07	0.95	0.95	0.95	0.95

(0.95: still in the ballistic start, t/τp = 30.) Across the intermediate range (1e-6 m, 5 seeds × 20 000 particles, corrected for the ballistic start), stock + analytical gives D_num/D = 1.00 ± 0.01 at dt/τp = 0.3, 3 and 100. Bounded + analytical gives 1.0, 1.55, 4.7 and 47 at dt/τp = 1, 3, 10 and 100, i.e. ≈ dt/(2τp). Defaults (since 2026-10-12): `brownian = "BrownianMotion"`, `u_integration = "analytical"`. The other choices still work, with a warning.

Is `analytical` safe for the other forces? All forces enter one linear equation per parcel and sub-step, m dU/dt = Sp (Uc − U) + Su, with Uc, Sp and Su frozen at the start of the sub-step. The drag is the implicit part Sp; gravity/buoyancy, lift, Brownian and pressure-gradient forces are explicit (Su). `analytical` is the exact solution of that frozen equation for any dt/τp; CrankNicolson and Euler only approximate it. Within this framework, analytical is never less accurate. Where it is limited, so are the other schemes:

- **Explicit velocity-dependent forces as strong as the drag.** Lift, Magnus or any Su ∝ (Uc − U) is lagged one step. For dt ≫ τp this is a fixed-point iteration with gain ≈ |∂F/∂U| τp/m, i.e. force/drag: it converges only if that is < 1. Saffman lift/drag ≈ 0.17 d √(G/ν), < 1e-2 here. It approaches 1 only for large particles in strong shear (e.g. 100 μm in water at G ~ 1e4 1/s). Remedy: dt < τp or an implicit formulation; the integrator does not help. - **Steep spatial variation within a sub-step.**

Near-wall or field forces (lubrication, thermophoresis in a thin layer, electric fields) are frozen at the sub-step start, and for dt ≫ τp the parcel takes the matching terminal velocity for the whole sub-step. The accuracy is set by maxCo and the mesh, not by the integrator. - **Re-dependent drag with large, changing slip** (heavy particles in sprays). C_D(Re) is frozen per sub-step, the same for every scheme. It does not matter at the particle Re here (≤ 1e-3). - **Models that use the instantaneous parcel velocity.** For dt ≫ τp, stock Brownian + analytical gives U − Uc = √(2D/dt) ξ: the mean velocity over the step, which is correct for displacement but smaller than the thermal velocity (U std/v_th = 0.79, 0.45 and 0.14 at dt/τp = 3, 10 and 100). Stick/escape and deposition counts are unaffected. Rebound restitution, erosion, impaction-velocity sticking criteria or particle–particle collision models would see the step-mean velocity, not the thermal one. - **Two-way coupling** (not used here): analytical returns the exact time-integrated momentum exchange of the frozen step to the fluid, so it stays consistent.

Gradual release (`manualInjection duration`, added to `src/myIntermediate` 2026-10-12). Optional, default 0, which keeps the old all-at-SOI behaviour. With duration > 0, the N positions of the file are released one by one in file order, position k at SOI + (k + ½)·duration/N. The index is computed from time, so it is the same on every processor and after a restart. The generator writes the positions in random order, so the release is spatially uniform at every moment. Test (A wedge, 5 × 1000 parcels, release over 1–3 s, written every 0.5 s): 250 parcels per injector every 0.5 s, all 1000 by t = 3 s, nothing after, ages spanning exactly the release window, and 1000 distinct trajectories per injector (no position released twice). With duration 0, all 5000 appear right after SOI, as before.

Latent bug, not fixed: with `outputByInjectorId`, localInteraction maps injector IDs with `injIdToIndex_.insert(id, nInjectors++)`, which increments even when an ID repeats. Two injection models sharing an `injectorID` would then index past the end of its output. The generator gives each diameter one injector, so it is not triggered.

ForceMagnitudes. Registered for nuclear parcels in `makeNuclearParcelCloudFunctionObjects.H` (2026-10-12). An older build of `liblagrangianMyIntermediate` (e.g. on the cluster) needs rebuilding, or set `force_magnitudes = False`.

Flow-field analysis (`flowAnalysis.py`)

```
python3 flowAnalysis.py cases/<case>        (OpenFOAM sourced; reconstructs a parallel case)
```

Samples the latest time on a meridional cut and horizontal cuts at z/H = 0.25, 0.5, 0.75, and writes to `<case>/analysis/` (contours in a rainbow colormap, `CMAP = "turbo"`):

- `meridional.png`: temperature, and speed with the streamlines of the axisymmetric stream function (equal mass-flow steps) carrying arrows in the flow direction (the radial axis is stretched);
- `radial.png`: U_z and T across the liquid at five heights;
- `axial.png`: T along z at five radii (inner wall or axis, ¼, ½, ¾, outer wall) with the axis thermocouples, and the strongest up- and down-flow;
- `sections.png`: T and U_z contours on the horizontal cuts, one colour range per row for all three heights (wedge: the radial profiles revolved through 360°, i.e. the 5° wedge repeated all around);
- `flowSummary.txt`: velocities, circulating mass flow, Re, Pe, Ra, boundary layer, and the variation around θ (3D).

Medium mesh with boundary layer, steady, converged (A 1281, B 1692 SIMPLE iterations), current defaults:

	A (annulus)	B (cylinder)
circulation	one toroidal cell: up along the inner ring, down along the cooled outer wall	one cell: up in the core, down along the cooled wall
strongest up / down	5.9 / 4.9 mm/s (at r ≈ 2.9 / 3.95 mm)	15.5 (on the axis) / 9.7 mm/s (0.5 mm from the wall)
circulating mass flow	146 mg/s (max at z ≈ 65 mm; ~10 mg/s near the bottom)	311 mg/s
turnover H / U_{max}	12.7 s	2.3 s
Re = U_{max} L / ν (L = gap / diameter)	10	105
Pe = U_{max} L / α	37	378
ΔT across the liquid at mid-height	98 K	83 K
down-flow layer at the outer wall	0.85 mm of 1.8 mm	1.3 mm of 3.5 mm
variation around θ (mid-radius)	≤ 0.14 K, ≤ 0.03 mm/s; $U_\theta \approx 0$	≤ 0.16 K, ≤ 0.07 mm/s; $U_\theta \approx 0$

Both flows are laminar and axisymmetric in the steady solution, which is what the wedge cases below exploit.

Axisymmetric wedge cases (`MESH_TYPE = "wedge"`)

Selectable in the questionnaire (section 3, "Mesh type?", with the wedge angle) or with `MESH_TYPE` in `caseDefaults.py`. `MESH_TYPE = "wedge"` builds a `WEDGE_ANGLE` (5°) wedge, one cell around, with `front`/`back` wedge patches; for B the axis is a collapsed edge (`axis` patch, empty, no faces). Same radial and axial cells as the 3D mesh, case name ending in `_wedge`. The processor count is asked in section 8 ("Parallel run"): the suggestion is 1 for the flow alone (a few thousand cells, serial is fastest) and 4 with particles, where the parcel tracking dominates and is split across the processors (`decomposePar`, `runParallel`, `reconstructPar` with the lagrangian fields). Boundary conditions given as total powers (power-mode fluxes, the ring flux, the absolute heat source) are scaled to the wedge; `heatBalance` scales back and reports full-capsule watts. The SIMPLE target for U is 5e-6 on wedges: the azimuthal component ($U \approx 0$) stalls at ~1e-6 while U_r, U_z, T and p_rgh converge fully.

(medium mesh, boundary layer, steady)	A 3D	A wedge	B 3D	B wedge
cells	407k	4.6k	185k	2.4k
iterations / time	1281 / ~25 min on 4 procs	790 / 13 s	1692 / ~15 min on 4 procs	1300 / 12 s
strongest up / down [mm/s]	5.91 / 4.87	5.91 / 4.87	15.5 / 9.73	15.5 / 9.76
circulating mass flow [mg/s]	146	146	311	309
ΔT across the liquid at mid-height [K]	97.6	98.0	82.7	83.0
mean salt T [K]	916.94	916.93	985.24	985.23

The wedge reproduces the steady 3D solutions. It cannot show 3D instabilities or a non-axisymmetric flow: check those once with the 3D transient stage. In A the circulation is weak in the lowest ~15 mm (heat crosses the gap there mostly by conduction) and strengthens with height; the turning layers sit under the free surface (U_r up to 1.9 mm/s outward) and at the bottom.

Mesh sensitivity (`meshStudy.py`)

```
python3 meshStudy.py run        (all levels, wedges, steady; then flowAnalysis and the GCI)
python3 meshStudy.py analyse    (tables and plot from finished runs)
```

Five levels per capsule (`coarse` … `xxfine`), every part of the mesh refined together by a ratio of ~1.4 per level: target cell sizes, first boundary-layer cell (30 µm at medium) and layer growth ratio ($1.15^{(h/h_{medium})}$). A: 2.1k → 33.5k cells, B: 1.1k → 17.3k. Grid Convergence Index (Celik et al. 2008) on the three finest levels; results in `cases/meshStudy/meshStudy.txt`, `meshStudy.csv`, `meshStudy.png`.

Quantity	A: medium vs finest	A: GCI	B: medium vs finest	B: GCI
mean / max salt T	0.007 % / 0.001 %	≤ 0.002 %	0.005 % / 0.017 %	≤ 0.004 %
strongest up-flow	0.24 %	0.01 %	0.67 %	0.04 %
strongest down-flow	0.06 %	0.34 %	0.26 %	3.1 % (p = 0.18, not asymptotic)
circulating mass flow	0.10 %	0.02 %	0.35 %	0.14 %
ΔT across the salt	0.001 %	~0	0.01 %	0.01 %
heat through the outer wall	0.02 %	0.02 %	0.002 %	~0
ring heat into the salt (A)	3.4 %	3.1 %	–	–

The temperature field is mesh-independent from the medium level on. Velocities change by < 1 % and are partly limited by the post-processing grid (60 × 240 points) that locates the extrema. The ring heat (1.6 W) is a small difference of ~250 W fluxes and the most sensitive quantity (0.05 W between medium and finest). The coarse levels stopped at the iteration cap without meeting the residual targets, but their values are close as well. The medium boundary-layer mesh is adequate; the wedges can afford `fine` or `xfine` at no real cost.

Output

- `postProcessing/heatBalance_<solver>.dat`: Q_{gen}, total and per-patch heat out [W] (conduction + radiation), $Q_{gen}-Q_{out}$ (storage), $T_{avg/min/max}$, the area-averaged T of each patch and, with radiation on, the radiative part Qrad of each patch. Use it for convergence, for `dT_internal` and for comparing with the thermocouples.
- `postProcessing/probes`: T and U at (r, 0, z) points (`PROBE_RADII` × `PROBE_HEIGHT_FRACTIONS`).
- `caseSummary.txt`: mesh, properties, and lumped estimates (mean steady T, time constant, Rayleigh number, thermal boundary-layer thickness and the cells across it).